

AI 掘金头条-缓存和模型调用

Redis 和 调用 QWen 大模型



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

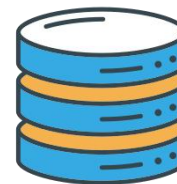
缓存：高速的临时数据存储机制



用户 / 前端



FastAPI 服务器
(业务逻辑控制中心)



数据库

1000人同时查
数据库会怎样



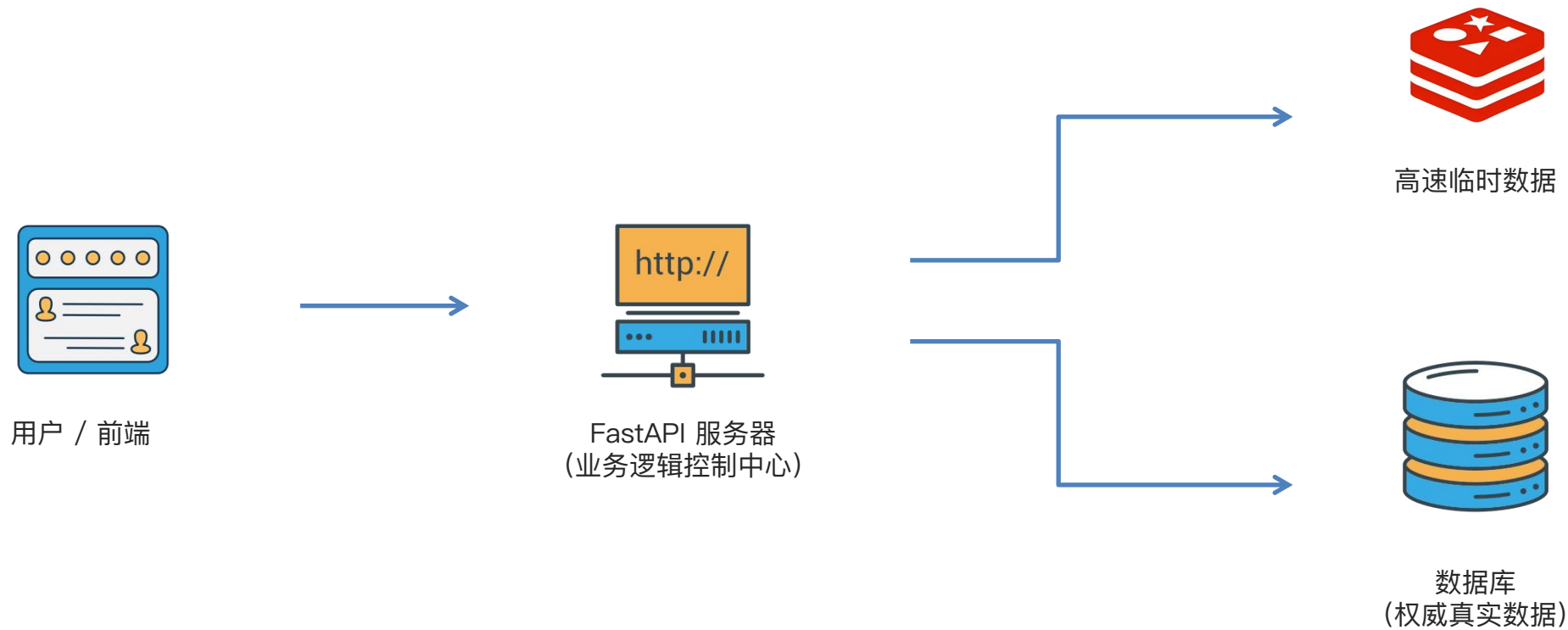
数据缓存

缓存是一种存储机制，用于临时存储数据或计算结果，当再次需要这些数据时，可以快速从缓存中检索，而不是重新进行耗时或昂贵的获取和计算过程。

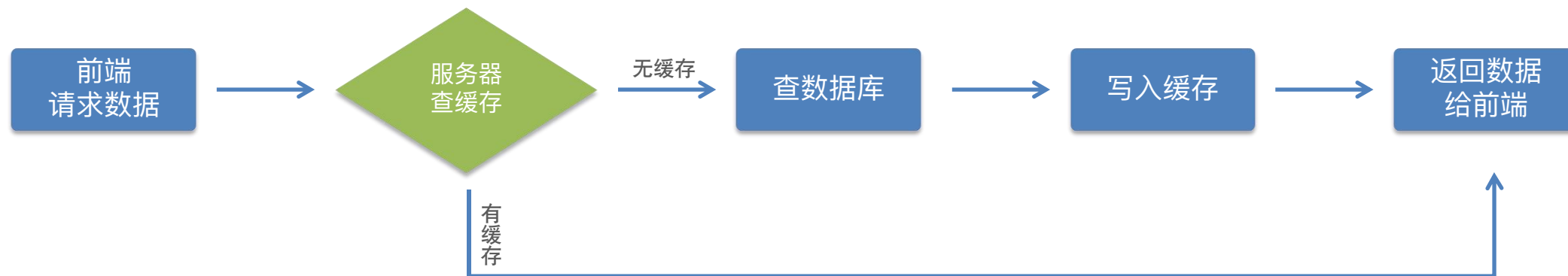
在网站开发中，缓存（Cache）是一个非常重要的概念，其核心作用是提高性能、降低延迟和减轻服务器负载。

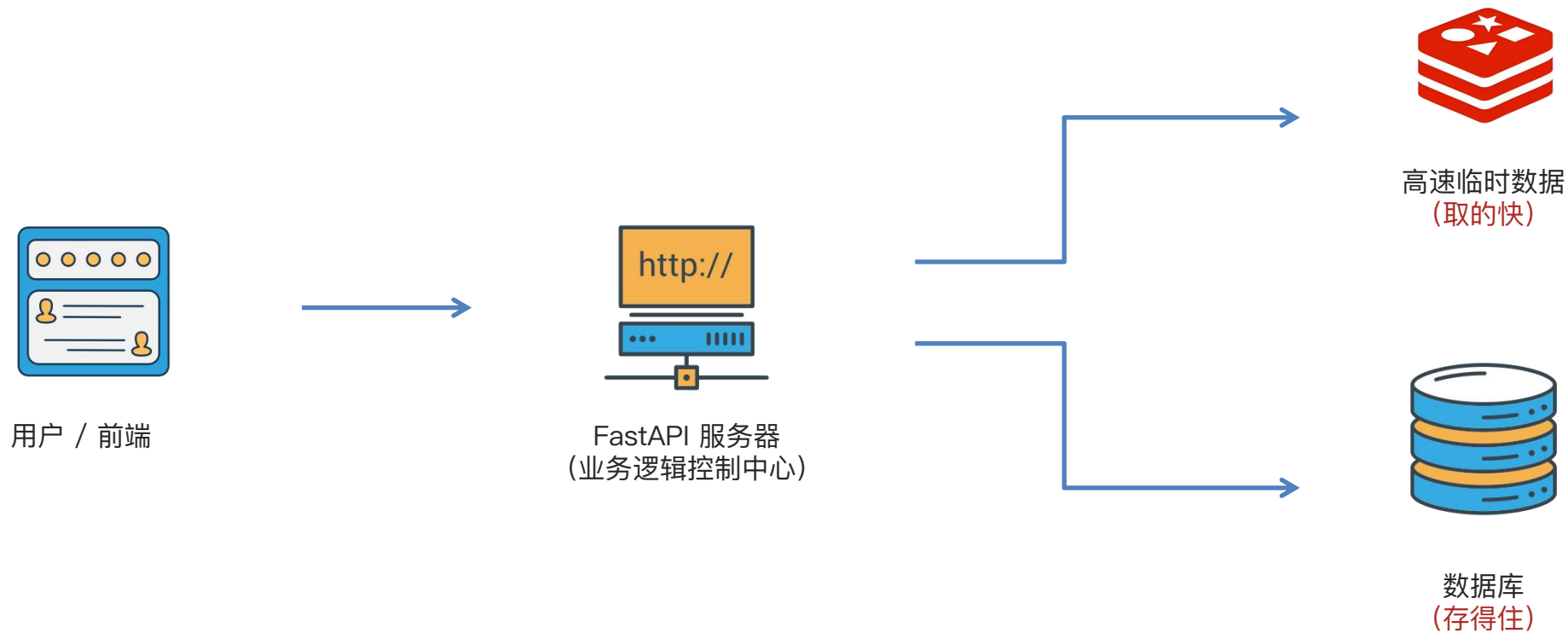
主要优势：

1. 提升性能和用户体验
2. 减轻服务器/数据库负载
3. 降低网络延迟
4. 节省资源和成本



项目中的缓存流程

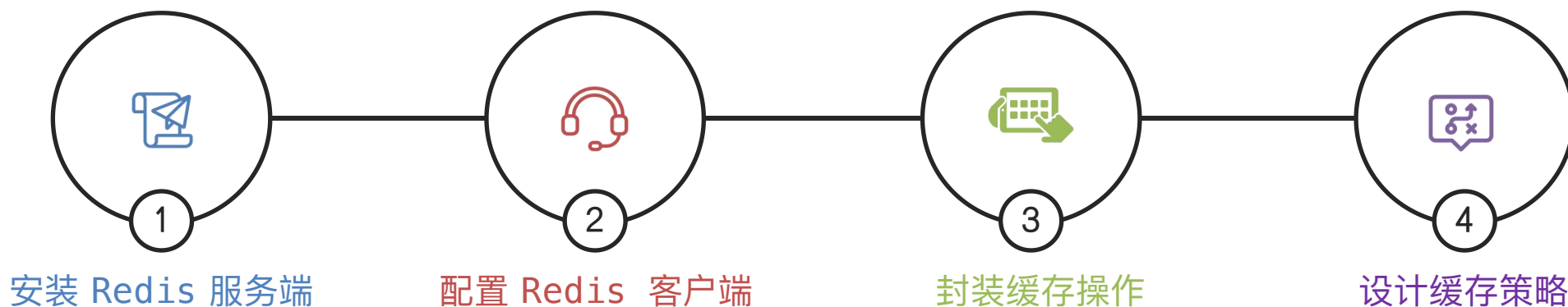




Redis

Redis 是一种高性能的 **Key-Value 存储系统**，它将数据存储在内存中，因此读写速度极快，非常适合作为应用层的缓存服务。

在 FastAPI 这样的后端框架中，通常在应用层使用像 **Redis** 这样的**内存数据存储**作为缓存。



安装 Redis 服务端

01

Mac

安装: `brew install redis`

验证: `redis-server`

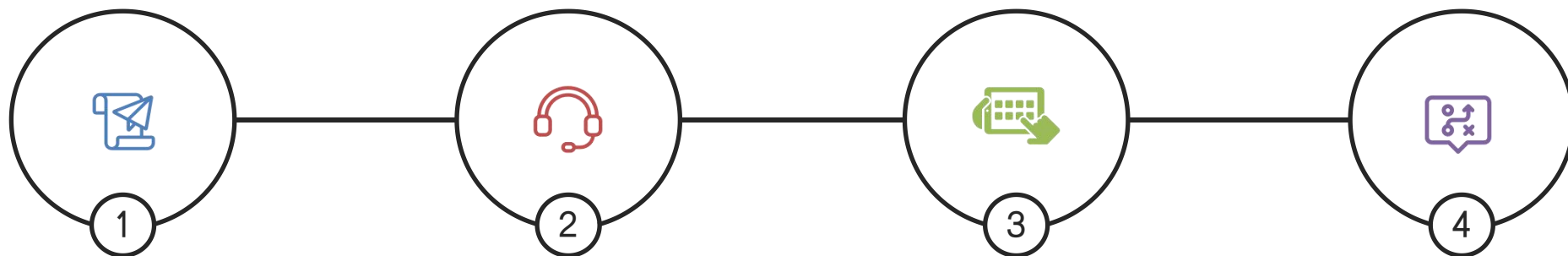
02

Windows

安装: 双击 .msi文件

验证: `redis-cli ping`

Redis



安装 Redis 服务端

1. 本地或服务器安装并启动 Redis 服务
2. 默认端口号 6379

配置 Redis 客户端

1. 安装 Redis 的客户端
2. 配置 Redis 客户端 (地址、端口等)

封装缓存操作

设计缓存策略

配置 Redis 客户端

01

安装 Redis 客户端

```
pip install redis
```

02

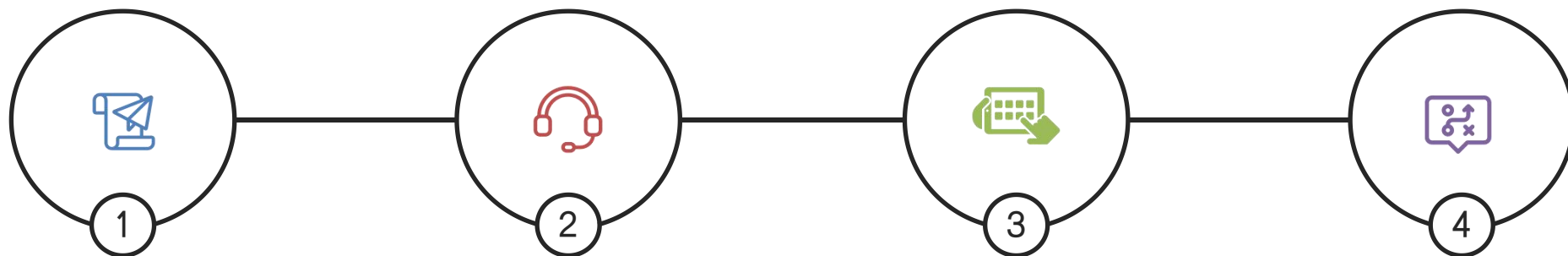
配置 Redis 客户端

```
import redis.asyncio as redis
```

```
redis.Redis(...)
```

- **host**: Redis 服务器地址
- **port**: 端口号 6379
- **db**: 数据库编号 (0~15)
- **decode_responses**: 是否将返回的数据从字节流解码为字符串

Redis



安装 Redis 服务端

1. 本地或服务器安装并启动 Redis 服务
2. 默认端口号 6379

配置 Redis 客户端

1. 安装 Redis 的客户端
2. 配置 Redis 客户端 (地址、端口等)

封装缓存操作

1. 设置缓存: `setex`
2. 获取缓存: `get`
3. 删除缓存: `delete`

设计缓存策略

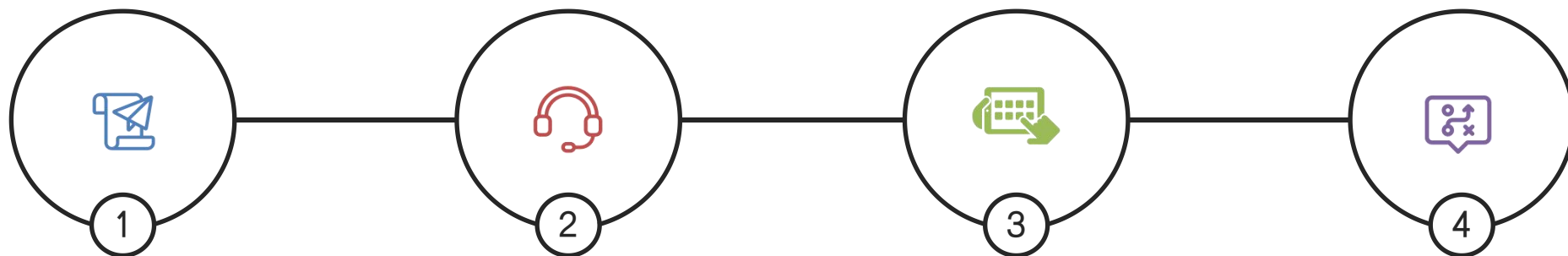
封装缓存操作

缓存操作就是围绕 Redis 做“存、取、删、判断、过期”等操作，让数据访问更快、数据库压力更小。

Redis 存储数据：key – value

方法	参数	描述
setex	key: str, expire: int (秒), value: str	设置缓存并指定过期时间 (秒)
get	key: str	获取缓存值。若缓存不存在，返回 None
delete	key: str	删除指定的缓存键
exists	key: str	检查缓存键是否存在，返回布尔值

Redis



安装 Redis 服务端

1. 本地或服务器安装并启动 Redis 服务
2. 默认端口号 6379

配置 Redis 客户端

1. 安装 Redis 的客户端
2. 配置 Redis 客户端 (地址、端口等)

封装缓存操作

1. 设置缓存: `setex`
2. 获取缓存: `get`
3. 删除缓存: `delete`

设计缓存策略

最常见的策略 – 旁路策略

- 读: 先查缓存, 有数据则返回, 没有数据则查询数据库
- 写: 更新数据库后, 更新或删除缓存数据

设计缓存策略

旁路缓存策略（Cache-Aside）是一种常见的缓存策略，其核心概念是应用程序主动管理缓存，在读取数据时先检查缓存，如果缓存中没有数据，则从数据库或其他数据源加载数据，并将数据存入缓存；当数据更新或删除时，应用程序也负责更新或删除缓存中的数据。

```
async def get_categories(db: AsyncSession, skip: int=0, limit: int=100):  
    # 尝试从缓存获取  
    cached_categories = await get_cached_categories()  
    if cached_categories:  
        return cached_categories  
    # 如果缓存没有，则从数据库中查询  
    stmt = select(Category).offset(skip).limit(limit)  
    result = await db.execute(stmt)  
    categories = result.scalars().all()  
    # 缓存新闻分类列表  
    if categories:  
        categories = jsonable_encoder(categories)  
        await cache_categories(categories)  
  
    return categories
```

设计缓存策略

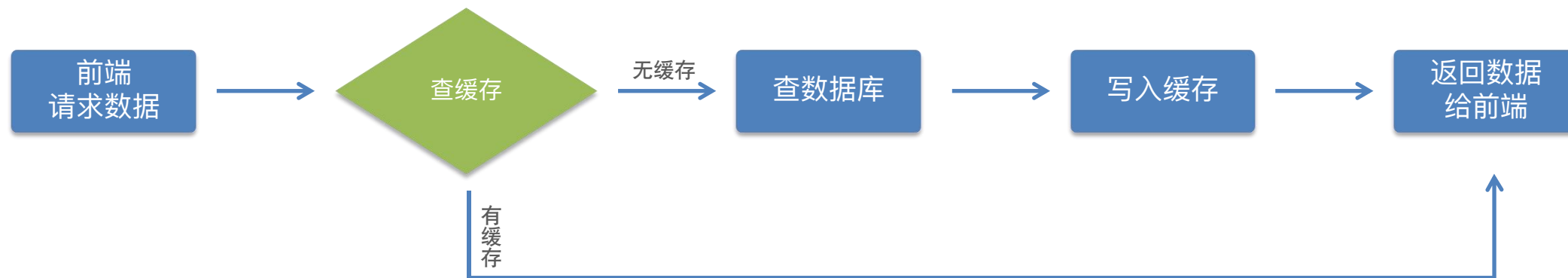
不同类型的数据，缓存时间不同，否则会出现**缓存雪崩**

数据越稳定，缓存越久；数据变化越快，缓存越短

类型	时间
分类、配置	7200（2小时）
列表数据	600（10分钟）
详情数据	1800（30分钟）
验证码	120（2分钟）

设计缓存策略 – 新闻列表

- 缓存 Key (唯一) : news:list:分类 ID:页码:每页数量
- 缓存 Value: 新闻列表



调用大模型



QWen 模型

在前端页面中有模型请求的接口指定，支持千问（Qwen）系列模型的使用，可以直接指定对应的API和key

使用步骤：

登录[阿里云百炼模型广场](#) → 选择模型（通义千问3-Max） → API 参考 → 获取 HTTP 请求地址 + API Key → 前端调

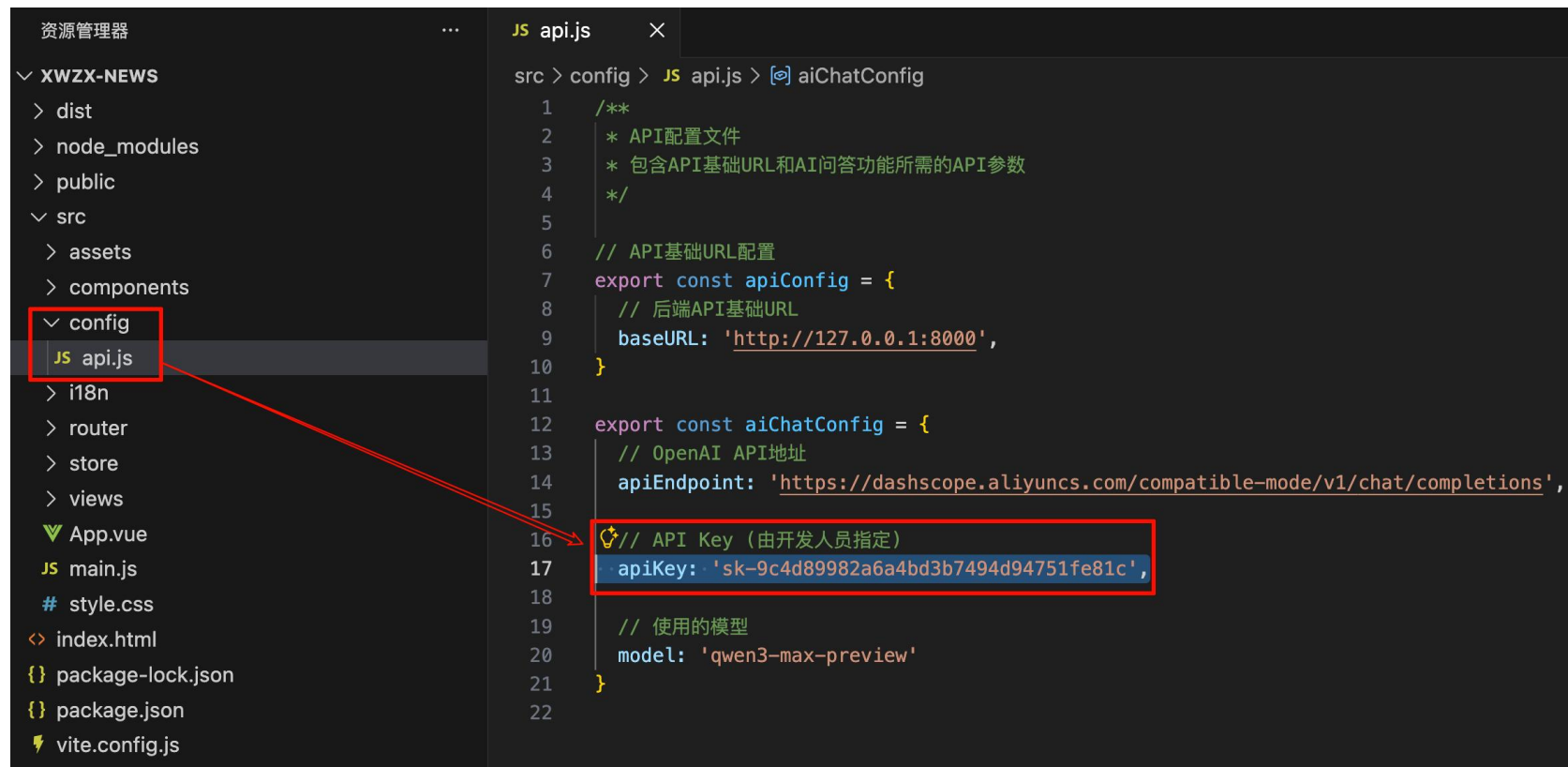
用



前端调用模型

前端项目 → src/ → config/ → api.js → 替换 大模型 HTTP 请求地址 + API Key + 模型

[阿里云百炼模型广场](#)



```
src > config > JS api.js > aiChatConfig
1  /**
2   * API配置文件
3   * 包含API基础URL和AI问答功能所需的API参数
4   */
5
6  // API基础URL配置
7  export const apiConfig = {
8    // 后端API基础URL
9    baseUrl: 'http://127.0.0.1:8000',
10 }
11
12 export const aiChatConfig = {
13   // OpenAI API地址
14   apiEndpoint: 'https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions',
15
16   // API Key (由开发人员指定)
17   apiKey: 'sk-9c4d89982a6a4bd3b7494d94751fe81c',
18
19   // 使用的模型
20   model: 'qwen3-max-preview'
21 }
22
```

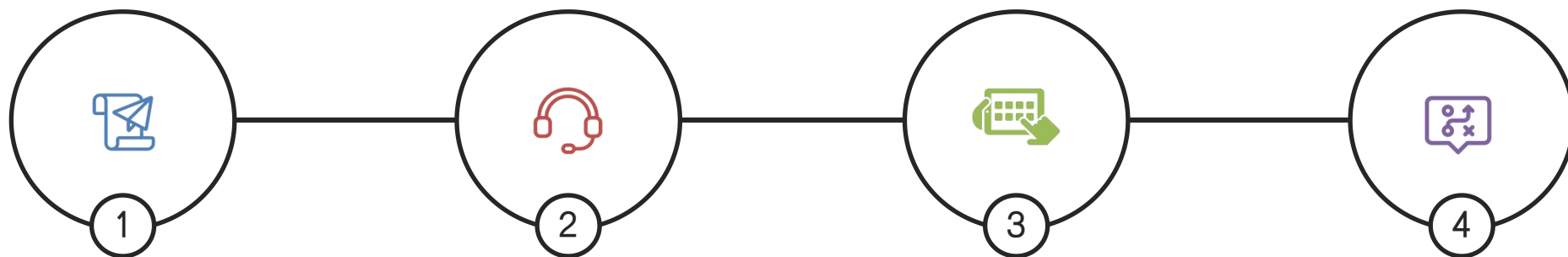
本章总结

这都是知识啊



本章总结

Redis 是一种高性能的 **Key-Value 存储系统**，它将数据存储在内内存中，因此读写速度极快，非常适合作为应用层的缓存服务。



安装 Redis 服务端

1. 本地或服务器安装并启动 Redis 服务
2. 默认端口号 6379

配置 Redis 客户端

1. 安装 Redis 的客户端
2. 配置 Redis 客户端 (地址、端口等)

封装缓存操作

1. 设置缓存: `setex`
2. 获取缓存: `get`
3. 删除缓存: `delete`

设计缓存策略

最常见的策略 – **旁路策略**

- 读: 先查缓存, 有数据则返回, 没有数据则查询数据库
- 写: 更新数据库后, 更新或删除缓存数据

模型训练与服务化

看好你奥





传智教育旗下高端IT教育品牌

